

Task on RDS

RDS Is Important for following reasons

1 Automated Backups & Restore

- RDS automatically takes daily snapshots.
- You can restore the DB to *any point in time*.
- No manual backups required like on EC2.

2 High Availability with Multi-AZ

- Provides **automatic failover** to another AZ.
- Ensures minimal downtime for production workloads.

3 Read Replicas for Scalability

- Offloads read traffic.
- Reduces load on primary DB.
- Improves application performance.

4 Easy Scaling

- You can increase storage and compute with a few clicks.

5 Security

- Built-in encryption, IAM authentication, VPC isolation, automatic patching.

6 Migration and Compatibility

- Supports MariaDB, MySQL, PostgreSQL, Oracle, SQL Server.
- Easy migration using native tools (mysqldump, AWS DMS).

TOOLS USED

1 EC2 Instance – Used to install MariaDB/MySQL, create databases, insert dummy data, and take backups using mysqldump.

2 MariaDB/MySQL CLI Tools – Used for database login, SQL queries, exporting and importing data.

3 AWS RDS – Used to create managed MariaDB/MySQL databases, enable Multi-AZ, automated backups, snapshots, and read replicas.

4 AWS Console & AWS CLI – Used to configure RDS instances, networking, connectivity, and perform migrations.

5 mysqldump & mysql client – Used for exporting data from EC2 and importing it into RDS.

- 6 **S3 (optional)** – Can store DB backups before migration.
- 7 **CloudWatch** – Used for monitoring RDS performance and health.

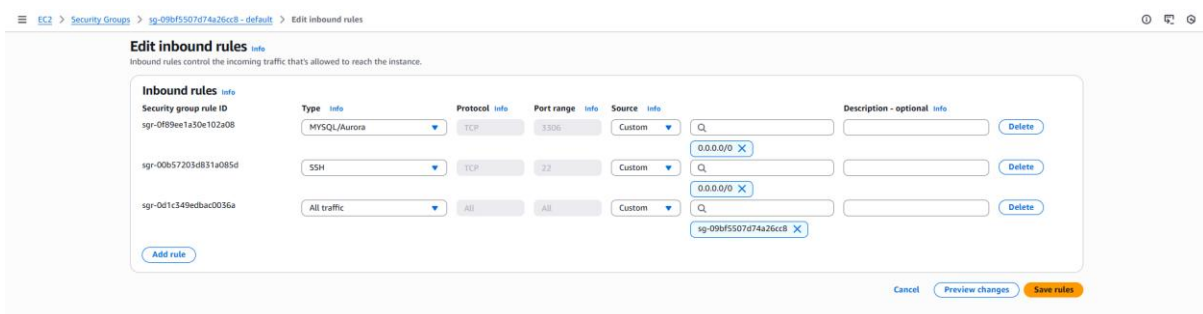
1. Create MariaDB DB on EC2.

STEPS:

Here are the **steps to install MariaDB and log in** on Amazon Linux 2023

- **Install MariaDB server:**
- `sudo dnf install mariadb105-server -y`
- **Start the MariaDB service:**
- `sudo systemctl start mariadb`
- **Enable service on boot:**
- `sudo systemctl enable mariadb`
- **Secure the installation:**
- `sudo mysql_secure_installation`
- **Log in to MariaDB:**
- `mysql -u root -p`

Launched one instance and given allow permission to mysql 3306 port number



Installed mariadb

```
[ec2-user@ip-12-0-0-219 ~]$ sudo dnf install mariadb105-server -y
Last metadata expiration check: 1 day, 11:21:04 ago on Wed Dec 10 08:00:45 2023.
Dependencies resolved.
```

Package	Architecture	Version	Repository	Size
Installing:				
mariadb105-server	x86_64	3:10.5.29-1.amzn2023.0.1	amazonlinux	10 M
Installing dependencies:				
mariadb-connector-c	x86_64	3.3.10-1.amzn2023.0.1	amazonlinux	215 k
mariadb-connector-c-config	noarch	3.3.10-1.amzn2023.0.1	amazonlinux	9.5 k
mariadb105	x86_64	3:10.5.29-1.amzn2023.0.1	amazonlinux	1.5 M
mariadb105-common	x86_64	3:10.5.29-1.amzn2023.0.1	amazonlinux	28 k
mariadb105-errmsg	x86_64	3:10.5.29-1.amzn2023.0.1	amazonlinux	212 k
mysql-sslinx	noarch	1.0.4-2.amzn2023.0.3	amazonlinux	36 k

Given password and setup mariadb

```

NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB
SERVERS IN PRODUCTION USE! PLEASE READ EACH STEP CAREFULLY!

In order to log into MariaDB to secure it, we'll need the current
password for the root user. If you've just installed MariaDB, and
haven't set the root password yet, you should just press enter here.

Enter current password for root (enter for none):
OK, successfully used password, moving on...

Setting the root password or using the unix socket ensures that nobody
can log into the MariaDB root user without the proper authorization.

You already have your root account protected, so you can safely answer 'n'.

Switch to unix socket authentication [Y/n] y
Enabled successfully!
Reloading privilege tables..
... Success!

You already have your root account protected, so you can safely answer 'n'.

Change the root password? [Y/n] y
New password:
Re-enter new password:
Password updated successfully!
Reloading privilege tables..
... Success!

By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them. This is intended only for testing, and to make the installation
go a bit smoother. You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n] y

```

Created mariadb and setup mariadb

```

CPU: 524ms
Group: /system.slice/mariadb.service
L:5058 /usr/libexec/mariadbd --basedir=/usr

Dec 11 09:22:19 ip-12-0-9-219.ec2.internal mariadb-prepare-db-dir[5015]: The second is mysqllocalhost, it has no password either, but
Dec 11 09:22:19 ip-12-0-9-219.ec2.internal mariadb-prepare-db-dir[5015]: you need to be the system 'mysql' user to connect.
Dec 11 09:22:19 ip-12-0-9-219.ec2.internal mariadb-prepare-db-dir[5015]: After connecting you can set the password, if you would need to be
Dec 11 09:22:19 ip-12-0-9-219.ec2.internal mariadb-prepare-db-dir[5015]: able to connect as any of these users with a password and without sudo
Dec 11 09:22:19 ip-12-0-9-219.ec2.internal mariadb-prepare-db-dir[5015]: See the MariaDB Knowledgebase at https://mariadb.com/kb
Dec 11 09:22:19 ip-12-0-9-219.ec2.internal mariadb-prepare-db-dir[5015]: Please report any problems at https://mariadb.org/jira
Dec 11 09:22:19 ip-12-0-9-219.ec2.internal mariadb-prepare-db-dir[5015]: The latest information about MariaDB is available at https://mariadb.org/.
Dec 11 09:22:19 ip-12-0-9-219.ec2.internal mariadb-prepare-db-dir[5015]: Consider joining MariaDB's strong and vibrant community:
Dec 11 09:22:19 ip-12-0-9-219.ec2.internal mariadb-prepare-db-dir[5015]: https://mariadb.org/get-involved/
Dec 11 09:22:20 ip-12-0-9-219.ec2.internal systemd[1]: Started mariadb.service - MariaDB 10.5 database server.
[ec2-user@ip-12-0-9-219 ~]$ mysql -u root -p
Enter password:
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MariaDB connection id is 13
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\n' to clear the current input statement.

```

2. Insert some dummy data.

Steps:

- Login to MariaDB: `sudo mysql -u root -p`
- Select DB: `USE company;`
- Insert data:
 - `INSERT INTO employees (name, role) VALUES ('John','Engineer'),('Alice','Manager'),('Bob','HR');`
 - Verify table: `SHOW TABLES;`
 - View data: `SELECT * FROM employees;`

Added dummy data as shown in the image

```

MariaDB [company]>
MariaDB [company]> INSERT INTO employees (name, role)
-> VALUES ('John', 'Engineer'),
-> ('Alice', 'Manager'),
-> ('Bob', 'HR');
Query OK, 3 rows affected (0.001 sec)
Records: 3 Duplicates: 0 Warnings: 0

MariaDB [company]> show table
-> show table^C
^C^C
show tables;
show tables;
-> show tables;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near "show tables" at line 2
MariaDB [company]> show tables;
+-----+
| Tables_in_company |
+-----+
| employees          |
+-----+

```

Check the all table using by entering select * from employees; it showing the data

```
MariaDB [company]> SELECT * FROM employees;
+----+-----+-----+
| id | name | role |
+----+-----+-----+
| 1 | John | Engineer |
| 2 | Alice | Manager |
| 3 | Bob | SA |
| 4 | John | Engineer |
| 5 | Alice | Manager |
| 6 | Bob | SA |
+----+-----+-----+
6 rows in set (0.000 sec)
```

3. Take the backup of dummy data on EC2.

Steps:

- Export the database using mysqldump:
- `mysqldump -u root -p company > /home/ec2-user/company_backup.sql`
- Verify the backup file exists:
- `ls -lh /home/ec2-user/company_backup.sql`

Backup was taken successfully.

```
ec2-user@ip-12-0-9-219 ~]$ mysqldump -u root -p company > /home/ec2-user/company_backup.sql
Enter password:
ec2-user@ip-12-0-9-219 ~]$ ls -lh /home/ec2-user/company_backup.sql
-rw-r--r--. 1 ec2-user ec2-user 2.1K Dec 11 10:22 /home/ec2-user/company_backup.sql
ec2-user@ip-12-0-9-219 ~]$
```

4. Launch MariaDB RDS instance.

Steps:

Go to AWS RDS Console → Create Database

- Engine: MariaDB
- Edition: Free tier (if possible)
- Instance class: db.t3.micro
- Storage: 20 GB
- Connectivity:
 - Choose correct VPC
 - Public access: YES (for testing)
 - Add EC2 Security Group to allow connectivity

AWS [Alt+S] United States (N. Virginia) Account ID: 6796-2572-2057 Arjmand

Aurora and RDS Subnet groups Create DB subnet group

Name
You won't be able to modify the name after your subnet group has been created.
MYSUBNET
Must contain from 1 to 255 characters. Alphanumeric characters, spaces, hyphens, underscores, and periods are allowed.

Description
workingRDS

VPC
Choose a VPC identifier that corresponds to the subnets you want to use for your DB subnet group. You won't be able to choose a different VPC identifier after your subnet group has been created.
My_vpc_varginaR (vpc-0018a135ed108a41b)
3 Subnets, 2 Availability Zones

Add subnets
Availability Zones
Choose the Availability Zones that include the subnets you want to add.
Choose an availability zone
us-east-1a X

Subnets
Choose the subnets that you want to add. The list includes the subnets in the selected Availability Zones.
Select subnets
pub_subnetN.1
Subnet ID: subnet-0c9115a216ca7844c CIDR: 12.0.0.0/22 X

For Multi-AZ DB clusters, you must select 3 subnets in 3 different Availability Zones.

Subnets selected (1)

Availability zone	Subnet name	Subnet ID	CIDR block

Created subnet group

AWS [Alt+S] United States (N. Virginia) Account ID: 6796-2572-2057 Arjmand

Aurora and RDS Subnet groups

Successfully created MYSUBNET. View subnet group

Subnet groups (1) Edit Delete Create DB subnet group

Filter by subnet group

☐	Name	Description	Status	VPC
☐	mysubnet	workingRDS	Complete	vpc-0018a135ed108a41b

Aurora and RDS

- Dashboard
- Databases
- Query editor
- Performance insights
- Snapshots
- Exports in Amazon S3
- Automated backups
- Reserved instances
- Proxies
- Subnet groups
- Parameter groups
- Option groups

Aurora and RDS Databases Create database

Create database

Choose a database creation method

☒ Full configuration
You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☐ Easy create
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Engine options

Engine type

☐ Aurora (MySQL Compatible)

☐ Aurora (PostgreSQL Compatible)

☐ MySQL

☐ PostgreSQL

☒ MariaDB

☐ Oracle

☐ Microsoft SQL Server

☐ IBM Db2

Engine version
View the engine versions that support the following database features.

☒ Hide filters

☒ Show only versions that support the Amazon RDS Optimized Writes
Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.

Engine version
MariaDB 11.8.5

Created a RDA database

Q Filter by databases

DB identifier

▲

Status

▼

Role

▼

Engine

▼

Upgrade rollout order

▼

Region ...

▼

Size

▼

Recommendations

▼

CPU

▼

Current activity

▼

Mainte...

▼

VPC

▼

○ database-1

⌚ Creating

Instance

MariaDB

SECOND

-

db.t3.micro

-

none

vpc-0018a

Enhanced Monitoring, CloudWatch Logs and DevOps Guru

Enhanced Monitoring

☐ Enable Enhanced monitoring

Enabling Enhanced Monitoring metrics are useful when you want to see how different processes or threads use the CPU.

Log exports

Select the log types to publish to Amazon CloudWatch Logs

☐ Audit log

☐ Error log

☐ General log

☐ iam-db-auth-error log

☐ Slow query log

IAM role

The following service-linked role is used for publishing logs to CloudWatch Logs.

RDS service-linked role

▼ Additional configuration

Database options, encryption turned on, backup turned off, backtrack turned off, maintenance, CloudWatch Logs, delete protection turned off.

Database options

Initial database name [info](#)

If you do not specify a database name, Amazon RDS does not create a database.

DB parameter group [info](#)

default:mariaadb11.8

Option group [info](#)

default:mariaadb11-8

Backup

☐ Enable automated backup

Create a point-in-time snapshot of your database

Backup tags [info](#)

Copy tags from the source database to the automated backup and snapshots respectively

☐ Copy tags to automated backup

This is a one-time setting. Future tag modifications need manual updates.

☒ Enable encryption

Choose to encrypt the given instance. Master key IDs and aliases appear in the list after they have been created using the AWS Key Management Service console. [info](#)

AWS KMS key [info](#)

(default) aws/rds

Account

679625722057

KMS key ID

alias/aws/rds

Maintenance

Auto minor version upgrade [info](#)

☐ Enable auto minor version upgrade

Enabling auto minor version upgrade will automatically upgrade your database minor version. For limitations and more details, see Automatically upgrading the minor engine version [documentation](#). [info](#)

Maintenance window [info](#)

Select the period you want pending modifications or maintenance applied to the database by Amazon RDS.

☐ Choose a window

☒ No preference

☐ Enable deletion protection

Protects the database from being deleted accidentally. While this option is enabled, you can't delete the database.

Estimated monthly costs

The Amazon RDS Free Tier is available to you for 12 months. Each calendar month, the free tier will allow you to use the Amazon RDS resources listed below for free:

- 750 hrs of Amazon RDS in a Single-AZ db.t2.micro, db.t3.micro or db.t4g.micro Instance.
- 20 GB of General Purpose Storage (SSD).
- 20 GB for automated backup storage and any user-initiated DB Snapshots.

[Learn more about AWS Free Tier.](#) [info](#)

When your free usage expires or if your application use exceeds the free usage tiers, you simply pay standard, pay-as-you-go service rates as described in the [Amazon RDS Pricing page.](#) [info](#)

ⓘ You are responsible for ensuring that you have all of the necessary rights for any third-party products or services that you use with AWS services.

Cancel

Create database

5. Migrate database from EC2 to RDS.

Steps:

Created database and inserted dummy data on EC2

Used MariaDB/MySQL commands to create DB, tables, and insert sample employee records.

2. Took a backup of the EC2 database

Ran mysqldump on EC2 to export the database into a .sql backup file.

3. Launched a new MariaDB RDS instance

Configured engine version, instance type, storage, networking, and master credentials.

4. Enabled SSL and connected EC2 → RDS securely

Downloaded Amazon RDS SSL certificate and connected using the --ssl-ca option.

5. Created the same database (company) on RDS

Logged into RDS and created the empty target database.

6. Restored the EC2 backup into RDS

Ran the MySQL command to import the .sql backup into the RDS database.

Command used-mysql -h database-1.c03quu6cw8z1.us-east-1.rds.amazonaws.com \

--ssl-ca=us-east-1-bundle.pem \

-u admin -p

```
MariaDB [(none)]> USE company;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [company]> SHOW TABLES;
+-----+
| Tables_in_company |
+-----+
| employees          |
+-----+
1 row in set (0.001 sec)

MariaDB [company]> SELECT * FROM employees;
+----+-----+-----+
| id | name | role |
+----+-----+-----+
| 1  | John | Engineer |
| 2  | Alice | Manager |
| 3  | Bob  | HR      |
| 4  | John | Engineer |
| 5  | Alice | Manager |
| 6  | Bob  | HR      |
+----+-----+-----+
6 rows in set (0.001 sec)

MariaDB [company]> |
```

6. Install MySQL DB on EC2.

Steps:

- **Update server** → sudo yum update -y.
- **Install MariaDB server** → sudo yum install mariadb-server -y.
- **Start the service** → sudo systemctl start mariadb.
- **Enable at boot** → sudo systemctl enable mariadb.
- **Secure & login** → sudo mysql_secure_installation and then mysql -u root -p.

[illegible]

7. Launch MySQL RDS image.

- Go to AWS RDS Console → Databases → Create database.
 - Select Standard Create and choose MySQL engine.
 - Choose Free Tier template (if eligible) and set DB instance identifier, username, and password.
 - Select DB instance class (e.g., db.t3.micro) and allocate storage.
 - Under Connectivity, select your VPC, choose public/private access, and allow MySQL port 3306 in the security group.
 - Click Create database and wait until the MySQL RDS instance status becomes Available.
-
- Go to **AWS Console** → **RDS** → **Create database**.
 - Choose **Standard Create** and select **MySQL** as the engine.
 - Pick a **version**, choose a **template** (Free tier/Dev/Test/Production).
 - Set **DB instance identifier**, **master username**, and **password**.
 - Choose your **DB instance class** (e.g., db.t3.micro) and configure **storage**.
 - Configure **Connectivity** (VPC, subnet group, public access yes/no, security group).
 - Click **Create database** — AWS launches your **MySQL RDS instance (image)**.

Amazon RDS > Databases > Create database

Create database

Choose a database creation method

☒ Full configuration

☐ Easy create

You set all of the configuration options, including ones for availability, security, backups, and maintenance.

Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Engine options

Engine type

☐ Aurora (MySQL Compatible)

☐ Aurora (PostgreSQL Compatible)

☒ MySQL

☐ PostgreSQL









☐ MariaDB

☐ Oracle

☐ Microsoft SQL Server

☐ IBM Db2









Edition

☒ MySQL Community

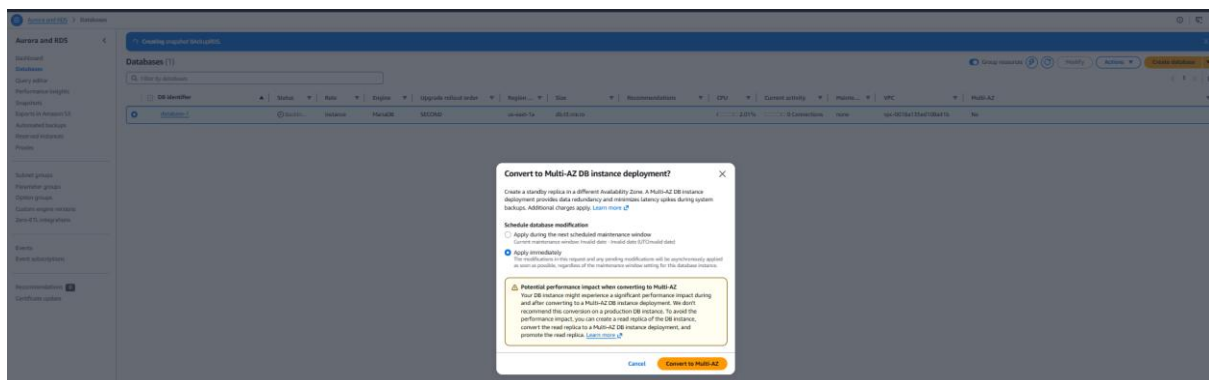
Engine version

8. Configure Multi-AZ.

Steps:

- Select RDS instance → Modify
- Choose **Multi-AZ (DB instance standby)**
- Apply immediately or during maintenance window

Modified RDA and changed to Multi_AZ



Resource ID db-4GBGHG6LEW74HGTETZMBD63OPY	IAM DB authentication Not enabled	Storage autoscaling Enabled
Created time December 11, 2025, 16:31 (UTC+05:30)	Multi-AZ Yes	Maximum storage threshold 1000 GiB
DB instance parameter group default.mariadb11.8 In sync	Secondary Zone us-east-1b	Storage file system configuration Current
Deletion protection Disabled		
Architecture settings Non-multitenant architecture		

9. Take backup of DB and restore the DB.

Steps:

- In the RDS console, select your MySQL/MariaDB RDS instance.
- Click Take Snapshot to create a full backup of the database.
- AWS saves this snapshot as a restorable image under RDS → Snapshots.
- To restore, open the snapshot and choose Restore Snapshot.
- AWS launches a new RDS DB instance using the data from that snapshot.
- After restoration, copy the new endpoint and test database access.
- Verify tables/data to ensure the restore was successful and matches the original.

Taken snapshot of the RDA

Take DB Snapshot

Preferences
To take a DB Snapshot, choose a database and name your DB Snapshot.

Snapshot type
☒ DB instance
☐ DB cluster

DB instance
DB instance identifier. This is the unique key that identifies a DB instance.
database-1

Snapshot name
Identifier for the DB Snapshot.
RDABACKUP

Snapshot identifier is case insensitive, but stored as all lower-case, as in "mysnapshot". Cannot be null, empty, or blank. Must contain from 1 to 255 alphanumeric characters or hyphens. First character must be a letter. Cannot end with a hyphen or contain two consecutive hyphens.

[Cancel](#) [Take snapshot](#)

Deleted the RDS as we can see

database-1

database-1

Summary

DB identifier database-1	Status Deleting	Role Instance	Engine MariaDB	Recommendations
CPU 4.25%	Class db.t3.micro	Current activity 0 Connections	Region & AZ us-east-1a	

[Connectivity & security](#) | [Monitoring](#) | [Logs & events](#) | [Configuration](#) | [Maintenance & backups](#) | [Data migrations](#) | [Tags](#) | [Recommendations](#)

Connectivity & security

[View details](#)

Delete database-1 instance

Permanently delete **database-1** DB instance. You can't undo this action.

Proceeding with this action will delete the instance with all its content and can affect related resources. [Learn more](#)

☐ **Create final snapshot**
Determines whether a final DB Snapshot is created before the DB instance is deleted.

☐ **Retain automated backups**
Determines whether retaining automated backups for 1 day after deletion

☒ **I acknowledge that upon instance deletion, automated backups, including system snapshots and point-in-time recovery, will no longer be available.**

To avoid accidental deletion provide additional written consent.

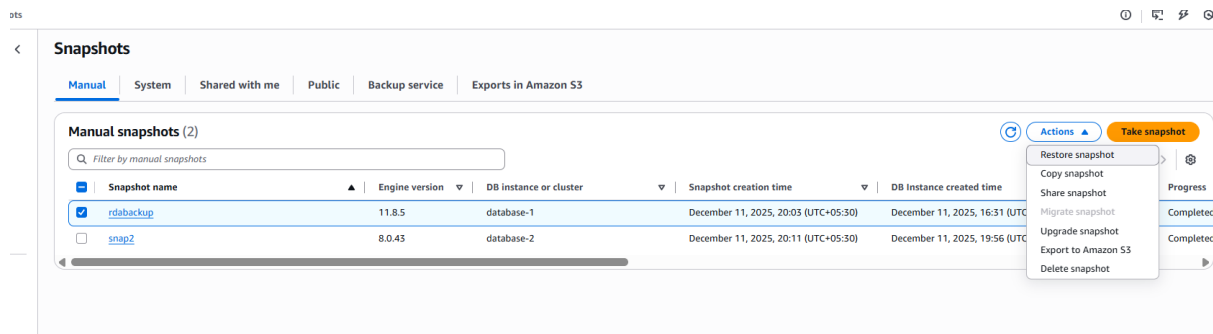
To confirm deletion, type *delete me* into the field.

delete me

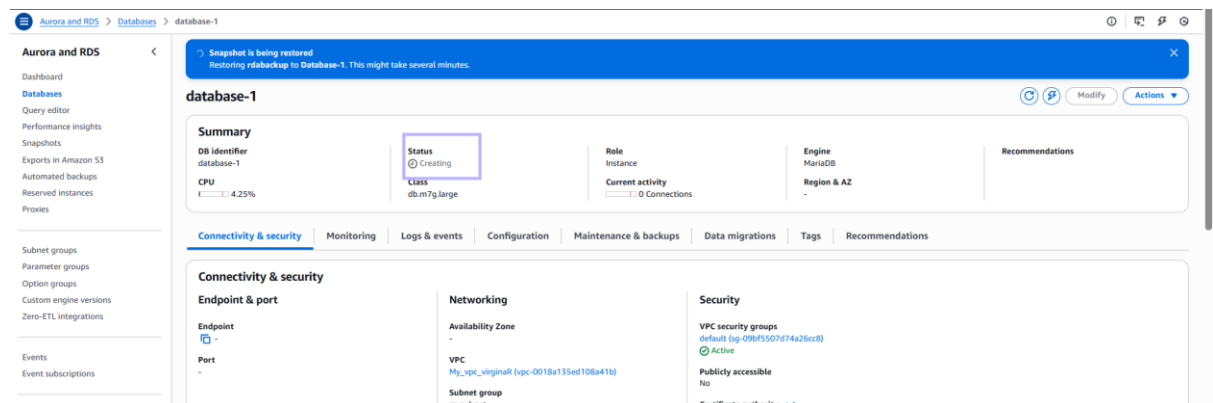
We strongly recommend taking a final snapshot before instance deletion since after your instance is deleted, automated backups will no longer be available.

[Cancel](#) [Delete](#)

Deleted RDS Recovering from the snapshot



Successfully recovered RDS from snapshot

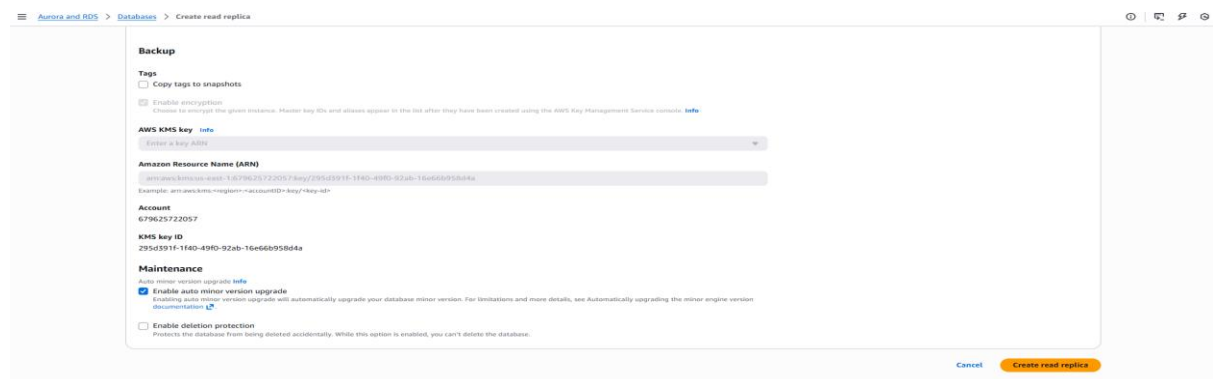


10. Create read replica.

Steps:

- Go to RDS → Databases and select your primary MySQL/MariaDB instance.
- Click Actions → Create Read Replica.
- Choose instance class, storage, and optionally a different Availability Zone.
- Create the replica and wait for the status to become available.
- Connect to the replica endpoint and verify it allows read-only queries only.

Creating replica by selecting snapshot and creating replica from actions



Replica is created successfully

Aurora and RDS

Databases

Query editor

Performance insights

Snapshots

Exports in Amazon S3

Automated backups

Reserved instances

Proxies

Creating replica rdsreplica in US East (N. Virginia)

Your database might take a few minutes to launch.

Databases (2)

Filter by databases

DB identifier

Status

Role

Engine

Upgrade rollout order

Region ...

Size

Recommendations

CPU

Current activity

database-1

Modify...

Primary

MariaDB

SECOND

us-east-1a

db.m7g.large

1.50%

0 Connec

rdsreplica

Creating

Replica

MariaDB

SECOND

us-east-1a

db.t3.micro

-